

RV32I Assembler

Generated by Doxygen 1.17.0

1 Data Structure Index	1
1.1 Data Structures	1
2 File Index	3
2.1 File List	3
3 Data Structure Documentation	5
3.1 array_impl Struct Reference	5
3.2 asm_t Struct Reference	5
3.3 instruction_t Struct Reference	6
3.4 master_array_impl Struct Reference	6
3.5 pair_t Struct Reference	6
3.6 table_impl Struct Reference	6
4 File Documentation	7
4.1 include/assembler.h File Reference	7
4.1.1 Detailed Description	7
4.1.2 Function Documentation	8
4.1.2.1 assemble()	8
4.2 assembler.h	8
4.3 include/dynamic_array.h File Reference	8
4.3.1 Detailed Description	9
4.3.2 Function Documentation	9
4.3.2.1 array_append()	9
4.3.2.2 array_create()	10
4.3.2.3 array_dupe()	10
4.3.2.4 array_free()	10
4.3.2.5 array_get()	11
4.3.2.6 array_get_size()	11
4.3.2.7 array_pop()	11
4.3.2.8 array_print()	12
4.3.2.9 array_set()	12
4.3.2.10 master_array_append()	12
4.3.2.11 master_array_create()	13
4.3.2.12 master_array_free()	13
4.3.2.13 master_array_print()	13
4.4 dynamic_array.h	13
4.5 include/instruction.h File Reference	14
4.5.1 Detailed Description	14
4.5.2 Function Documentation	15
4.5.2.1 instruction_lookup()	15
4.6 instruction.h	15
4.7 include/register.h File Reference	15

4.7.1 Detailed Description	16
4.7.2 Function Documentation	16
4.7.2.1 register_lookup()	16
4.8 register.h	16
4.9 include/table.h File Reference	16
4.9.1 Detailed Description	17
4.9.2 Function Documentation	17
4.9.2.1 table_create()	17
4.9.2.2 table_free()	18
4.9.2.3 table_get()	18
4.9.2.4 table_print()	18
4.9.2.5 table_set()	19
4.10 table.h	19
4.11 src/ assembler.c File Reference	19
4.11.1 Detailed Description	20
4.11.2 Function Documentation	20
4.11.2.1 assemble()	20
4.12 src/ dynamic_array.c File Reference	20
4.12.1 Detailed Description	21
4.12.2 Function Documentation	21
4.12.2.1 array_append()	21
4.12.2.2 array_create()	22
4.12.2.3 array_dupe()	22
4.12.2.4 array_free()	22
4.12.2.5 array_get()	23
4.12.2.6 array_get_size()	23
4.12.2.7 array_pop()	23
4.12.2.8 array_print()	24
4.12.2.9 array_set()	24
4.12.2.10 master_array_append()	24
4.12.2.11 master_array_create()	25
4.12.2.12 master_array_free()	25
4.12.2.13 master_array_print()	25
4.13 src/ instruction.c File Reference	25
4.13.1 Detailed Description	26
4.13.2 Function Documentation	26
4.13.2.1 instruction_lookup()	26
4.14 src/ main.c File Reference	26
4.14.1 Detailed Description	26
4.15 src/ register.c File Reference	27
4.15.1 Detailed Description	27
4.15.2 Function Documentation	27

4.15.2.1 register_lookup()	27
4.16 src/table.c File Reference	27
4.16.1 Detailed Description	28
4.16.2 Function Documentation	28
4.16.2.1 table_create()	28
4.16.2.2 table_free()	29
4.16.2.3 table_get()	29
4.16.2.4 table_print()	29
4.16.2.5 table_set()	30
Index	31

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

array_impl	5
asm_t	5
instruction_t	6
master_array_impl	6
pair_t	6
table_impl	6

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

include/ assembler.h	
Dynamically allocated string array and 2D master array interfaces	7
include/ dynamic_array.h	
Dynamically allocated string array and 2D master array interfaces	8
include/ instruction.h	
Instruction type and table for easy lookup	14
include/ register.h	
This library provides the functionality of lookup table for register values	15
include/ table.h	
Linear lookup table with string keys and integer values	16
src/ assembler.c	
Implementation for assembler.h	19
src/ dynamic_array.c	
Implementation for dynamic_array.h	20
src/ instruction.c	
Implementation for instruction.h	25
src/ main.c	
Assemble and bitstream assembly to FPGA	26
src/ register.c	
Implementation for register.h	27
src/ table.c	
Implementation for table.h	27

Chapter 3

Data Structure Documentation

3.1 array_impl Struct Reference

Data Fields

- int **capacity**
- int **size**
- char ** **data**

The documentation for this struct was generated from the following file:

- [src/dynamic_array.c](#)

3.2 asm_t Struct Reference

Data Fields

- const char * **filename**
- FILE * **file**
- array_t **assembly**
- master_array_t **clean_assembly**
- table_t **const_table**
- table_t **data_table**
- table_t **text_table**
- char * **data_image**
- char * **instructions**

The documentation for this struct was generated from the following file:

- [src/assembler.c](#)

3.3 `instruction_t` Struct Reference

Data Fields

- `const char * name`
- `const instr_type_t type`
- `const int opcode`
- `const int funct3`
- `const int funct7`

The documentation for this struct was generated from the following file:

- `include/instruction.h`

3.4 `master_array_impl` Struct Reference

Data Fields

- `int capacity`
- `int size`
- `array_t * arrays`

The documentation for this struct was generated from the following file:

- `src/dynamic_array.c`

3.5 `pair_t` Struct Reference

Data Fields

- `char * key`
- `int value`

The documentation for this struct was generated from the following file:

- `include/table.h`

3.6 `table_impl` Struct Reference

Data Fields

- `int capacity`
- `int size`
- `pair_t * pairs`

The documentation for this struct was generated from the following file:

- `src/table.c`

Chapter 4

File Documentation

4.1 include/assembler.h File Reference

Dynamically allocated string array and 2D master array interfaces.

Functions

- void [assemble](#) (const char *filename)

Reads the .asm file provided and produces data initialization and instruction initialization files.

4.1.1 Detailed Description

Dynamically allocated string array and 2D master array interfaces.

Author

Hoyt Darnell

Date

2026-06-27

4.1.2 Function Documentation

4.1.2.1 `assemble()`

```
void assemble (
    const char * filename)
```

Reads the .asm file provided and produces data initialization and instruction initialization files.

Parameters

<code>filename</code>	String of the .asm filename.
-----------------------	------------------------------

4.2 `assembler.h`

[Go to the documentation of this file.](#)

```
00001
00007
00008 #ifndef ASSEMBLER_H
00009 #define ASSEMBLER_H
00010
00015 void assemble(const char *filename);
00016
00017 #endif // ASSEMBLER_H
```

4.3 `include/dynamic_array.h` File Reference

Dynamically allocated string array and 2D master array interfaces.

Typedefs

- typedef struct `array_impl` * `array_t`
- typedef struct `master_array_impl` * `master_array_t`

Functions

- `array_t array_create` (int initial_capacity)
Creates an array_t type.
- `array_t array_dup` (array_t array)
Creates a deep copy of a given array_t.
- void `array_append` (array_t array, const char *str)
Appends a str to a given array_t.
- char * `array_pop` (array_t array)
Pops the last str off of an array.
- void `array_free` (array_t array)
Frees all elements in the given array.
- void `array_print` (array_t array, FILE *file)
Prints the contents of array.
- `master_array_t master_array_create` (int initial_capacity)
Creates a master_array_t type.
- void `master_array_append` (master_array_t master, array_t sub_array)

- Appends an array_t to a given master_array_t.*
 - void [master_array_free](#) (master_array_t master)
Frees all elements in the given master array.
 - void [master_array_print](#) (master_array_t master, FILE *file)
Prints the contents of master array.
 - void [array_set](#) (array_t array, const char *str, int index)
Sets the value of array at index to str.
 - char * [array_get](#) (const array_t array, int index)
Retrieves the value of array at index.
 - int [array_get_size](#) (const array_t array)
Retrieves the size of array.

4.3.1 Detailed Description

Dynamically allocated string array and 2D master array interfaces.

Author

Hoyt Darnell

Date

2026-06-27

4.3.2 Function Documentation

4.3.2.1 array_append()

```
void array_append (  
    array_t array,  
    const char * str)
```

Appends a str to a given array_t.

Parameters

	<i>array</i>	Array to append to.
in	<i>str</i>	String to append onto array.

Note

String is not directly appended a deep copy is instead appended.

4.3.2.2 array_create()

```
array_t array_create (  
    int initial_capacity)
```

Creates an array_t type.

Parameters

<i>initial_capacity</i>	Initial capacity of created array.
-------------------------	------------------------------------

Returns

Returns a pointer to the created dynamic array.

4.3.2.3 array_dupe()

```
array_t array_dupe (  
    array_t array)
```

Creates a deep copy of a given array_t.

Parameters

<i>array</i>	Array to create a deep copy of.
--------------	---------------------------------

Returns

Returns a pointer to the deep copy of array.

4.3.2.4 array_free()

```
void array_free (  
    array_t array)
```

Frees all elements in the given array.

Parameters

<i>array</i>	Array to free.
--------------	----------------

4.3.2.5 array_get()

```
char * array_get (  
    const array_t array,  
    int index)
```

Retrieves the value of array at index.

Parameters

<i>array</i>	Array to retrieve from.
<i>index</i>	Index of array to retrieve from.

Returns

Pointer to string requested.

4.3.2.6 array_get_size()

```
int array_get_size (  
    const array_t array)
```

Retrieves the size of array.

Parameters

in	<i>array</i>	Array to retrieve size from.
----	--------------	------------------------------

Returns

Size of array.

4.3.2.7 array_pop()

```
char * array_pop (  
    array_t array)
```

Pops the last str off of an array.

Parameters

<i>array</i>	Array to remove str from.
--------------	---------------------------

Returns

Returns the popped string.

4.3.2.8 array_print()

```
void array_print (
    array_t array,
    FILE * file)
```

Prints the contents of array.

Parameters

<i>array</i>	Array to print.
<i>file</i>	File to print to.

4.3.2.9 array_set()

```
void array_set (
    array_t array,
    const char * str,
    int index)
```

Sets the value of array at index to str.

Parameters

<i>array</i>	Array to set.
<i>str</i>	String to set.
<i>index</i>	Index of array to set.

4.3.2.10 master_array_append()

```
void master_array_append (
    master_array_t master,
    array_t sub_array)
```

Appends an array_t to a given master_array_t.

Parameters

<i>master</i>	Master array to append to.
<i>sub_array</i>	Array to append onto master array.

4.3.2.11 master_array_create()

```
master_array_t master_array_create (  
    int initial_capacity)
```

Creates a master_array_t type.

Parameters

<i>initial_capacity</i>	Initial capacity of created master array.
-------------------------	---

Returns

Returns a pointer to the created master array.

4.3.2.12 master_array_free()

```
void master_array_free (  
    master_array_t master)
```

Frees all elements in the given master array.

Parameters

<i>master</i>	Master array to free.
---------------	-----------------------

4.3.2.13 master_array_print()

```
void master_array_print (  
    master_array_t master,  
    FILE * file)
```

Prints the contents of master array.

Parameters

<i>master</i>	Master array to print.
<i>file</i>	File to print to.

4.4 dynamic_array.h

[Go to the documentation of this file.](#)

```
00001  
00007  
00008 #ifndef DYNAMIC_ARRAY_H  
00009 #define DYNAMIC_ARRAY_H  
00010  
00011  
00012 struct array_impl;  
00013 typedef struct array_impl* array_t;  
00014
```

```

00015 struct master_array_impl;
00016 typedef struct master_array_impl* master_array_t;
00017
00023 array_t array_create(int initial_capacity);
00024
00030 array_t array_dupes(array_t array);
00031
00038 void array_append(array_t array, const char *str);
00039
00045 char *array_pop(array_t array);
00046
00051 void array_free(array_t array);
00052
00058 void array_print(array_t array, FILE *file);
00059
00065 master_array_t master_array_create(int initial_capacity);
00066
00072 void master_array_append(master_array_t master, array_t sub_array);
00073
00078 void master_array_free(master_array_t master);
00079
00085 void master_array_print(master_array_t master, FILE *file);
00086
00093 void array_set(array_t array, const char *str, int index);
00094
00101 char *array_get(const array_t array, int index);
00102
00108 int array_get_size(const array_t array);
00109
00110 #endif // DYNAMIC_ARRAY_H

```

4.5 include/instruction.h File Reference

Instruction type and table for easy lookup.

Data Structures

- struct [instruction_t](#)

Enumerations

- enum [instr_type_t](#) {
R, **I**, **S**, **B**,
U, **J**}

Functions

- const [instruction_t](#) * [instruction_lookup](#) (const char *name)
Returns the [instruction_t](#) of a given RV32I instruction.

4.5.1 Detailed Description

Instruction type and table for easy lookup.

Author

Hoyt Darnell

Date

2026-06-27

4.5.2 Function Documentation

4.5.2.1 instruction_lookup()

```
const instruction\_t * instruction_lookup (
    const char * name)
```

Returns the [instruction_t](#) of a given RV32I instruction.

Parameters

<i>name</i>	Name of RV32I instruction.
-------------	----------------------------

Returns

Returns the [instruction_t](#) of the provided instruction.

4.6 instruction.h

[Go to the documentation of this file.](#)

```
00001
00007
00008 #ifndef INSTRUCTION_H
00009 #define INSTRUCTION_H
00010
00011 typedef enum { R, I, S, B, U, J } instr_type_t;
00012
00013 typedef struct {
00014     const char *name;
00015     const instr_type_t type;
00016     const int opcode;
00017     const int funct3;
00018     const int funct7;
00019 } instruction_t;
00020
00026 const instruction_t *instruction_lookup(const char *name);
00027
00028 #endif // INSTRUCTION_H
```

4.7 include/register.h File Reference

This library provides the functionality of lookup table for register values.

Functions

- int [register_lookup](#) (const char *key)
Searches the table and returns the value associated with the key.

4.7.1 Detailed Description

This library provides the functionality of lookup table for register values.

Author

Hoyt Darnell

Date

2026-06-27

4.7.2 Function Documentation

4.7.2.1 register_lookup()

```
int register_lookup (  
    const char * key)
```

Searches the table and returns the value associated with the key.

Parameters

key	String associated with the register (x1, gp, ...).
-----	--

Returns

Returns the integer value associated with the register.

4.8 register.h

[Go to the documentation of this file.](#)

```
00001  
00007  
00008 #ifndef REGISTER_H  
00009 #define REGISTER_H  
00010  
00016 int register_lookup(const char *key);  
00017  
00018 #endif // REGISTER_H
```

4.9 include/table.h File Reference

Linear lookup table with string keys and integer values.

```
#include <stdio.h>
```

Data Structures

- struct [pair_t](#)

Typedefs

- typedef struct [table_impl](#) * [table_t](#)

Functions

- [table_t table_create](#) (int initial_capacity)
Creates a table_t type.
- void [table_free](#) (table_t table)
Frees all elements of a table_t type.
- void [table_print](#) (const table_t table, FILE *file)
Prints the contents of a table.
- void [table_set](#) (table_t table, const char *key, int value)
Sets the value of key in the table.
- int [table_get](#) (const table_t table, const char *key, int *value)
Retrieves the value of a table associated with key.

4.9.1 Detailed Description

Linear lookup table with string keys and integer values.

Author

Hoyt Darnell

Date

2026-06-27

4.9.2 Function Documentation

4.9.2.1 table_create()

```
table_t table_create (  
    int initial_capacity)
```

Creates a table_t type.

Parameters

<i>initial_capacity</i>	Initial capacity of created table.
-------------------------	------------------------------------

Returns

Returns a pointer to the created table.

4.9.2.2 table_free()

```
void table_free (
    table_t table)
```

Frees all elements of a table_t type.

Parameters

<i>table</i>	Pointer to table to free.
--------------	---------------------------

4.9.2.3 table_get()

```
int table_get (
    const table_t table,
    const char * key,
    int * value)
```

Retrieves the value of a table associated with key.

Parameters

<i>table</i>	Pointer to table object.
<i>key</i>	Key to retrieve value from the table.
<i>value</i>	Pointer where the retrieved value will be stored.

Returns

Returns 1 if key is found 0 if not

4.9.2.4 table_print()

```
void table_print (
    const table_t table,
    FILE * file)
```

Prints the contents of a table.

Parameters

<i>table</i>	Pointer to table object.
<i>file</i>	File to print contents to.

4.9.2.5 table_set()

```
void table_set (
    table_t table,
    const char * key,
    int value)
```

Sets the value of key in the table.

Parameters

<i>table</i>	Pointer to table_t object.
<i>key</i>	Key of table in which the value will be changed.
<i>value</i>	Value to change key to be associated with.

4.10 table.h

[Go to the documentation of this file.](#)

```
00001
00007
00008 #ifndef TABLE_H
00009 #define TABLE_H
00010
00011 #include <stdio.h>
00012
00013 typedef struct {
00014     char *key;
00015     int value;
00016 } pair_t;
00017
00018 struct table_impl;
00019 typedef struct table_impl* table_t;
00020
00026 table_t table_create(int initial_capacity);
00027
00032 void table_free(table_t table);
00033
00039 void table_print(const table_t table, FILE *file);
00040
00047 void table_set(table_t table, const char *key, int value);
00048
00056 int table_get(const table_t table, const char *key, int *value);
00057
00058 #endif // TABLE_H
```

4.11 src/assembler.c File Reference

Implementation for [assembler.h](#).

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "../include/assembler.h"
#include "../include/dynamic_array.h"
#include "../include/table.h"
#include "../include/instruction.h"
#include "../include/register.h"
```

Data Structures

- struct [asm_t](#)

Macros

- `#define MAX_LINE_LENGTH 512`

Functions

- void [assemble](#) (const char *filename)
Reads the .asm file provided and produces data initialization and instruction initialization files.

4.11.1 Detailed Description

Implementation for [assembler.h](#).

4.11.2 Function Documentation

4.11.2.1 [assemble\(\)](#)

```
void assemble (
    const char * filename)
```

Reads the .asm file provided and produces data initialization and instruction initialization files.

Parameters

<i>filename</i>	String of the .asm filename.
-----------------	------------------------------

4.12 src/dynamic_array.c File Reference

Implementation for [dynamic_array.h](#).

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "../include/dynamic_array.h"
```

Data Structures

- struct [array_impl](#)
- struct [master_array_impl](#)

Typedefs

- typedef struct [array_impl](#) [array_impl_t](#)
- typedef struct [master_array_impl](#) [master_array_impl_t](#)

Functions

- `array_t array_create` (int initial_capacity)
Creates an array_t type.
- `array_t array_dup` (array_t array)
Creates a deep copy of a given array_t.
- `void array_append` (array_t array, const char *str)
Appends a str to a given array_t.
- `char * array_pop` (array_t array)
Pops the last str off of an array.
- `void array_free` (array_t array)
Frees all elements in the given array.
- `void array_print` (const array_t array, FILE *file)
Prints the contents of array.
- `void array_set` (array_t array, const char *str, int index)
Sets the value of array at index to str.
- `char * array_get` (const array_t array, int index)
Retrieves the value of array at index.
- `int array_get_size` (const array_t array)
Retrieves the size of array.
- `master_array_t master_array_create` (int initial_capacity)
Creates a master_array_t type.
- `void master_array_append` (master_array_t master, array_t sub_array)
Appends an array_t to a given master_array_t.
- `void master_array_free` (master_array_t master)
Frees all elements in the given master array.
- `void master_array_print` (master_array_t master, FILE *file)
Prints the contents of master array.

4.12.1 Detailed Description

Implementation for `dynamic_array.h`.

4.12.2 Function Documentation

4.12.2.1 array_append()

```
void array_append (
    array_t array,
    const char * str)
```

Appends a str to a given array_t.

Parameters

	<i>array</i>	Array to append to.
in	<i>str</i>	String to append onto array.

Note

String is not directly appended a deep copy is instead appended.

4.12.2.2 array_create()

```
array_t array_create (  
    int initial_capacity)
```

Creates an array_t type.

Parameters

<i>initial_capacity</i>	Initial capacity of created array.
-------------------------	------------------------------------

Returns

Returns a pointer to the created dynamic array.

4.12.2.3 array_dupe()

```
array_t array_dupe (  
    array_t array)
```

Creates a deep copy of a given array_t.

Parameters

<i>array</i>	Array to create a deep copy of.
--------------	---------------------------------

Returns

Returns a pointer to the deep copy of array.

4.12.2.4 array_free()

```
void array_free (  
    array_t array)
```

Frees all elements in the given array.

Parameters

<i>array</i>	Array to free.
--------------	----------------

4.12.2.5 array_get()

```
char * array_get (  
    const array_t array,  
    int index)
```

Retrieves the value of array at index.

Parameters

<i>array</i>	Array to retrieve from.
<i>index</i>	Index of array to retrieve from.

Returns

Pointer to string requested.

4.12.2.6 array_get_size()

```
int array_get_size (  
    const array_t array)
```

Retrieves the size of array.

Parameters

in	<i>array</i>	Array to retrieve size from.
----	--------------	------------------------------

Returns

Size of array.

4.12.2.7 array_pop()

```
char * array_pop (  
    array_t array)
```

Pops the last str off of an array.

Parameters

<i>array</i>	Array to remove str from.
--------------	---------------------------

Returns

Returns the popped string.

4.12.2.8 array_print()

```
void array_print (
    array_t array,
    FILE * file)
```

Prints the contents of array.

Parameters

<i>array</i>	Array to print.
<i>file</i>	File to print to.

4.12.2.9 array_set()

```
void array_set (
    array_t array,
    const char * str,
    int index)
```

Sets the value of array at index to str.

Parameters

<i>array</i>	Array to set.
<i>str</i>	String to set.
<i>index</i>	Index of array to set.

4.12.2.10 master_array_append()

```
void master_array_append (
    master_array_t master,
    array_t sub_array)
```

Appends an array_t to a given master_array_t.

Parameters

<i>master</i>	Master array to append to.
<i>sub_array</i>	Array to append onto master array.

4.12.2.11 master_array_create()

```
master_array_t master_array_create (  
    int initial_capacity)
```

Creates a master_array_t type.

Parameters

<i>initial_capacity</i>	Initial capacity of created master array.
-------------------------	---

Returns

Returns a pointer to the created master array.

4.12.2.12 master_array_free()

```
void master_array_free (  
    master_array_t master)
```

Frees all elements in the given master array.

Parameters

<i>master</i>	Master array to free.
---------------	-----------------------

4.12.2.13 master_array_print()

```
void master_array_print (  
    master_array_t master,  
    FILE * file)
```

Prints the contents of master array.

Parameters

<i>master</i>	Master array to print.
<i>file</i>	File to print to.

4.13 src/instruction.c File Reference

Implementation for [instruction.h](#).

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include "../include/instruction.h"
```

Functions

- const [instruction_t](#) * [instruction_lookup](#) (const char *name)
Returns the [instruction_t](#) of a given RV32I instruction.

4.13.1 Detailed Description

Implementation for [instruction.h](#).

4.13.2 Function Documentation

4.13.2.1 [instruction_lookup\(\)](#)

```
const instruction\_t * instruction_lookup (
    const char * name)
```

Returns the [instruction_t](#) of a given RV32I instruction.

Parameters

<i>name</i>	Name of RV32I instruction.
-------------	----------------------------

Returns

Returns the [instruction_t](#) of the provided instruction.

4.14 src/main.c File Reference

Assemble and bitstream assembly to FPGA.

```
#include <stdio.h>
#include <stdlib.h>
#include "../include/assembler.h"
```

Functions

- int **main** (int argc, char *argv[])

4.14.1 Detailed Description

Assemble and bitstream assembly to FPGA.

Author

Hoyt Darnell

Date

2026-06-27

4.15 src/register.c File Reference

Implementation for [register.h](#).

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "../include/table.h"
#include "../include/register.h"
```

Functions

- int [register_lookup](#) (const char *key)
Searches the table and returns the value associated with the key.

4.15.1 Detailed Description

Implementation for [register.h](#).

4.15.2 Function Documentation

4.15.2.1 register_lookup()

```
int register_lookup (
    const char * key)
```

Searches the table and returns the value associated with the key.

Parameters

key	String associated with the register (x1, gp, ...).
-----	--

Returns

Returns the integer value associated with the register.

4.16 src/table.c File Reference

Implementation for [table.h](#).

```
#include <stdlib.h>
#include <string.h>
#include "../include/table.h"
```

Data Structures

- struct [table_impl](#)

Typedefs

- typedef struct [table_impl](#) [table_impl_t](#)

Functions

- [table_t](#) [table_create](#) (int initial_capacity)
Creates a table_t type.
- void [table_free](#) (table_t table)
Frees all elements of a table_t type.
- void [table_print](#) (const table_t table, FILE *file)
Prints the contents of a table.
- void [table_set](#) (table_t table, const char *key, int value)
Sets the value of key in the table.
- int [table_get](#) (const table_t table, const char *key, int *value)
Retrieves the value of a table associated with key.

4.16.1 Detailed Description

Implementation for [table.h](#).

4.16.2 Function Documentation

4.16.2.1 [table_create\(\)](#)

```
table_t table_create (  
    int initial_capacity)
```

Creates a table_t type.

Parameters

<i>initial_capacity</i>	Initial capacity of created table.
-------------------------	------------------------------------

Returns

Returns a pointer to the created table.

4.16.2.2 table_free()

```
void table_free (
    table_t table)
```

Frees all elements of a table_t type.

Parameters

<i>table</i>	Pointer to table to free.
--------------	---------------------------

4.16.2.3 table_get()

```
int table_get (
    const table_t table,
    const char * key,
    int * value)
```

Retrieves the value of a table associated with key.

Parameters

<i>table</i>	Pointer to table object.
<i>key</i>	Key to retrieve value from the table.
<i>value</i>	Pointer where the retrieved value will be stored.

Returns

Returns 1 if key is found 0 if not

4.16.2.4 table_print()

```
void table_print (
    const table_t table,
    FILE * file)
```

Prints the contents of a table.

Parameters

<i>table</i>	Pointer to table object.
<i>file</i>	File to print contents to.

4.16.2.5 `table_set()`

```
void table_set (  
    table_t table,  
    const char * key,  
    int value)
```

Sets the value of key in the table.

Parameters

<i>table</i>	Pointer to <code>table_t</code> object.
<i>key</i>	Key of table in which the value will be changed.
<i>value</i>	Value to change key to be associated with.

Index

array_append
 dynamic_array.c, [21](#)
 dynamic_array.h, [9](#)
array_create
 dynamic_array.c, [21](#)
 dynamic_array.h, [9](#)
array_dupe
 dynamic_array.c, [22](#)
 dynamic_array.h, [10](#)
array_free
 dynamic_array.c, [22](#)
 dynamic_array.h, [10](#)
array_get
 dynamic_array.c, [22](#)
 dynamic_array.h, [10](#)
array_get_size
 dynamic_array.c, [23](#)
 dynamic_array.h, [11](#)
array_impl, [5](#)
array_pop
 dynamic_array.c, [23](#)
 dynamic_array.h, [11](#)
array_print
 dynamic_array.c, [23](#)
 dynamic_array.h, [11](#)
array_set
 dynamic_array.c, [24](#)
 dynamic_array.h, [12](#)
asm_t, [5](#)
assemble
 assembler.c, [20](#)
 assembler.h, [8](#)
assembler.c
 assemble, [20](#)
assembler.h
 assemble, [8](#)

dynamic_array.c
 array_append, [21](#)
 array_create, [21](#)
 array_dupe, [22](#)
 array_free, [22](#)
 array_get, [22](#)
 array_get_size, [23](#)
 array_pop, [23](#)
 array_print, [23](#)
 array_set, [24](#)
 master_array_append, [24](#)
 master_array_create, [24](#)
 master_array_free, [25](#)
 master_array_print, [25](#)

dynamic_array.h
 array_append, [9](#)
 array_create, [9](#)
 array_dupe, [10](#)
 array_free, [10](#)
 array_get, [10](#)
 array_get_size, [11](#)
 array_pop, [11](#)
 array_print, [11](#)
 array_set, [12](#)
 master_array_append, [12](#)
 master_array_create, [12](#)
 master_array_free, [13](#)
 master_array_print, [13](#)

include/assembler.h, [7](#), [8](#)
include/dynamic_array.h, [8](#), [13](#)
include/instruction.h, [14](#), [15](#)
include/register.h, [15](#), [16](#)
include/table.h, [16](#), [19](#)
instruction.c
 instruction_lookup, [26](#)
instruction.h
 instruction_lookup, [15](#)
instruction_lookup
 instruction.c, [26](#)
 instruction.h, [15](#)
instruction_t, [6](#)

master_array_append
 dynamic_array.c, [24](#)
 dynamic_array.h, [12](#)
master_array_create
 dynamic_array.c, [24](#)
 dynamic_array.h, [12](#)
master_array_free
 dynamic_array.c, [25](#)
 dynamic_array.h, [13](#)
master_array_impl, [6](#)
master_array_print
 dynamic_array.c, [25](#)
 dynamic_array.h, [13](#)

pair_t, [6](#)

register.c
 register_lookup, [27](#)
register.h
 register_lookup, [16](#)

- register_lookup
 - register.c, [27](#)
 - register.h, [16](#)
- src/assembler.c, [19](#)
- src/dynamic_array.c, [20](#)
- src/instruction.c, [25](#)
- src/main.c, [26](#)
- src/register.c, [27](#)
- src/table.c, [27](#)
- table.c
 - table_create, [28](#)
 - table_free, [28](#)
 - table_get, [29](#)
 - table_print, [29](#)
 - table_set, [29](#)
- table.h
 - table_create, [17](#)
 - table_free, [17](#)
 - table_get, [18](#)
 - table_print, [18](#)
 - table_set, [18](#)
- table_create
 - table.c, [28](#)
 - table.h, [17](#)
- table_free
 - table.c, [28](#)
 - table.h, [17](#)
- table_get
 - table.c, [29](#)
 - table.h, [18](#)
- table_impl, [6](#)
- table_print
 - table.c, [29](#)
 - table.h, [18](#)
- table_set
 - table.c, [29](#)
 - table.h, [18](#)